

Algorithms: Theory, Design and Implementation

5SENG003C

Name: S.M.P.M.Jayasinghe

IIT No:20241670

UOW:w2151908

1. Choice of Data Structure and Algorithm Data structure-Adjacency list using LinkedList Array

The graph is represented using the SimpleDirectedGraph class, which stores edges as an array of LinkedList objects. Each index in the array corresponds to a vertex, and the linked list at that index holds all vertices that the vertex points to (its outgoing neighbors). A vertex is considered deleted when its list is set to null.

Operation	Complexity	How it works
AddEdge(u,v)	$O(1)$	Append v to adj[u]
Findsink()	$O(V)$	Scans all non-null lists for an empty one
removeVertex(v)	$O(V+E)$	Sets adj[v]=null, scans all lists to remove v
countRemaining()	$O(V)$	Count non-null entries in the array.
findCycle()DFS	$O(V+E)$	Recursive DFS with path tracking.

The adjacency list was chosen over an adjacency matrix because it uses less memory for sparse graphs ($O(V + E)$ vs $O(V^2)$) and naturally supports the operations needed for sink elimination. The LinkedList allows $O(1)$ appending when adding edges, and efficient removal of specific elements when cleaning up incoming edges during vertex deletion.

Task 3- GraphParser Class

Graph input is handled by a dedicated GraphParser class, keeping parsing logic separate from the algorithm. The parser reads the input file using the Java Scanner class. The first integer on the first line is the vertex count, which is used to initialise the graph. Every subsequent pair of integers on each line represents a directed edge from vertex u to vertex v. Two graph objects are populated simultaneously. One for sink elimination and one backup copy for cycle detection. The parser correctly handles any valid input file in the format described in the cw specification.

2.Algorithm-Sink elimination

The sink-elimination algorithm is based on the key property that every finite acyclic directed graph must contain at least one sink — a vertex with no outgoing edges. If every vertex had at least one outgoing edge, any path would eventually revisit a vertex, forming a cycle. The algorithm proceeds as follows:

- 1) If the graph is empty → return YES (acyclic).
- 2) If no sink is found → return NO (cycle exists).
- 3) Remove a sink and all edges pointing to it, then repeat.

When a cycle is detected (step 2), the original graph copy is searched using a recursive DFS. The DFS tracks the current path and reports a cycle when it revisits a vertex already on the path.

Example A — Acyclic graph (a_40_0.txt, 40 vertices)

The algorithm removes sinks one by one until the graph is empty, confirming it is acyclic.

```
Graph loaded from: C:\Users\ASUS\Downloads\acyclicity_source - Copy\benchmarks\acyclic\40_0.txt
Vertices: 40

=== Sink-elimination algorithm ===
Starting vertex count: 40
Removing sink: 24 (vertices remaining after removal: 39)
Removing sink: 19 (vertices remaining after removal: 38)
Removing sink: 1 (vertices remaining after removal: 37)
Removing sink: 8 (vertices remaining after removal: 36)
Removing sink: 23 (vertices remaining after removal: 35)
Removing sink: 22 (vertices remaining after removal: 34)
Removing sink: 2 (vertices remaining after removal: 33)
Removing sink: 17 (vertices remaining after removal: 32)
Removing sink: 34 (vertices remaining after removal: 31)
Removing sink: 6 (vertices remaining after removal: 30)
Removing sink: 35 (vertices remaining after removal: 29)
Removing sink: 30 (vertices remaining after removal: 28)
Removing sink: 25 (vertices remaining after removal: 27)
Removing sink: 29 (vertices remaining after removal: 26)
Removing sink: 39 (vertices remaining after removal: 25)
Removing sink: 36 (vertices remaining after removal: 24)
Removing sink: 15 (vertices remaining after removal: 23)
Removing sink: 4 (vertices remaining after removal: 22)
Removing sink: 26 (vertices remaining after removal: 21)
Removing sink: 9 (vertices remaining after removal: 20)
Removing sink: 27 (vertices remaining after removal: 19)
Removing sink: 32 (vertices remaining after removal: 18)
```

```
Removing sink: 18 (vertices remaining after removal: 17)
Removing sink: 0 (vertices remaining after removal: 16)
Removing sink: 20 (vertices remaining after removal: 15)
Removing sink: 33 (vertices remaining after removal: 14)
Removing sink: 21 (vertices remaining after removal: 13)
Removing sink: 10 (vertices remaining after removal: 12)
Removing sink: 7 (vertices remaining after removal: 11)
Removing sink: 28 (vertices remaining after removal: 10)
Removing sink: 31 (vertices remaining after removal: 9)
Removing sink: 5 (vertices remaining after removal: 8)
Removing sink: 12 (vertices remaining after removal: 7)
Removing sink: 14 (vertices remaining after removal: 6)
Removing sink: 37 (vertices remaining after removal: 5)
Removing sink: 16 (vertices remaining after removal: 4)
Removing sink: 11 (vertices remaining after removal: 3)
Removing sink: 38 (vertices remaining after removal: 2)
Removing sink: 13 (vertices remaining after removal: 1)
Removing sink: 3 (vertices remaining after removal: 0)
Graph is now empty.
Result: YES (acyclic)

Answer: YES - the graph is acyclic.
```

Example B — Cyclic graph (c_40_0.txt, 40 vertices)

Sinks are removed until none remain, then DFS reports the cycle.

```
Graph loaded from: C:\Users\ASUS\Downloads\acyclicity_source - Copy\benchmarks\cyclic\c_40_0.txt
Vertices: 40

=== Sink-elimination algorithm ===
Starting vertex count: 40
Removing sink: 27 (vertices remaining after removal: 39)
Removing sink: 14 (vertices remaining after removal: 38)
Removing sink: 19 (vertices remaining after removal: 37)
Removing sink: 36 (vertices remaining after removal: 36)
Removing sink: 1 (vertices remaining after removal: 35)
Removing sink: 38 (vertices remaining after removal: 34)
Removing sink: 11 (vertices remaining after removal: 33)
Removing sink: 15 (vertices remaining after removal: 32)
Removing sink: 30 (vertices remaining after removal: 31)
Removing sink: 5 (vertices remaining after removal: 30)
No sink found - graph contains a cycle.
Result: NO (not acyclic)

=== Cycle detection (DFS) ===
Cycle found: 0 -> 6 -> 29 -> 4 -> 13 -> 32 -> 26 -> 17 -> 10 -> 9 -> 37 -> 7 -> 39 -> 25 -> 3 -> 24 -> 10

Answer: NO - the graph contains a cycle.
```

3. Performance Analysis

Theoretical complexity

Let V be the number of vertices and E the number of edges. The findSink method scans all V array slots each time it is called, costing $O(V)$ per call. In the worst case (acyclic graph) it is called V times, giving

$O(V^2)$ total for sink finding. The `removeVertex` method scans all adjacency lists to remove incoming edges, costing $O(V + E)$ per call and $O(V^2 + VE)$ total across all removals. For the benchmark graphs where E is roughly proportional to V (sparse graphs), this simplifies to $O(V^2)$ overall. The DFS cycle detection runs once in $O(V + E)$ time. Space complexity is $O(V + E)$ for storing the adjacency list.

Empirical data — doubling experiment

The algorithm was timed against both acyclic and cyclic benchmark sets (5 files per size, averaged). Times are in milliseconds

Vertices(V)	Acyclic avg(ms)	Cyclic avg(ms)	Doubling ratio(acyclic)
40	0.11	0.06	---
80	0.20	0.13	1.74
160	0.47	0.28	2.34
320	1.40	1.21	3.00
640	4.39	2.93	3.14

Interpretation

At small sizes the doubling ratio is close to 2, consistent with near-linear behaviour. As the graph grows, the ratio approaches 4, which is consistent with quadratic $O(V^2)$ growth. This matches the theoretical analysis: the repeated $O(V)$ scan in `findSink` across V removals dominates the runtime. The cyclic benchmarks are consistently faster because the algorithm terminates early as soon as no sink is found, often well before all vertices are processed.

Big-O Summary

Time complexity: $O(V^2 + VE)$ — dominated by the repeated `findSink` scan and the $O(V + E)$ cost of `removeVertex` called V times. For sparse graphs ($E = O(V)$) this simplifies to $O(V^2)$.

Space complexity: $O(V + E)$ — the adjacency list stores each vertex and each directed edge exactly once. The implementation could be improved to $O(V + E)$ overall by maintaining a queue of sinks updated incrementally (as in Kahn's topological sort algorithm), eliminating the repeated full scan